

CONTENTS

- Overview
- 1 Why the same question gets different answers from different models
- 2 RAG: retrieval, augmentation, generation
- 3 Embeddings: finding documents that share no words with the query
- 4 What RAG costs: latency per query, infrastructure forever
- 5 Fine-tuning: continuing training on a narrow dataset
- 6 How the weights actually move
- 7 The bill for fine-tuning: data, GPUs, maintenance and forgetting
- 8 Prompt engineering: aiming attention at capability that is already there
- 9 Where prompt engineering stops
- 10 Combining all three
- Glossary
- Check yourself
- Where to go next

RAG, Fine-Tuning and Prompt Engineering: Choosing How to Improve an LLM

Know which of the three levers to pull, what each one actually costs, and how to layer them.

For engineers building on top of language models who have to decide where the effort goes · 26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

[Watch the source video](#)

[Read the full lesson](#)

[Read the highlights](#)

BEFORE YOU START

- What a large language model does at the input/output level: tokens in, tokens out
- Comfort reading Python
- A working idea of what a vector is — a fixed-length list of numbers

Overview

A language model's knowledge is frozen at the moment training stopped, and it lives inside the weights rather than in any file you can open. Ask about something newer than the cutoff, or something private to your organisation, and the model has nothing to work from — so it guesses, fluently.

Three techniques address this, and the useful way to tell them apart is by where in the stack they intervene. RAG changes what reaches the model at query time. Fine-tuning changes the weights. Prompt engineering changes the request. That is the whole taxonomy: retrieval supplies facts, fine-tuning installs behaviour, prompting redirects capability that is already present.

They are not competitors, and picking between them is mostly a matter of naming what is actually missing. Missing facts point at retrieval. Wrong behaviour, tone or format points at fine-tuning. A capable model answering a badly specified question points at the prompt. Production systems normally use all three, layered.

KEY TAKEAWAYS

- ✓ Parametric knowledge is fixed at the training cutoff — anything newer or private has to arrive through the prompt.
- ✓ RAG is retrieve, augment, generate: the weights are untouched, only the model's input changes.
- ✓ Retrieval matches meaning rather than words, because both the query and the corpus are converted to vectors and compared geometrically.
- ✓ RAG's price is latency on every single query plus an embedding-and-indexing pipeline you have to keep in sync forever.
- ✓ Fine-tuning updates weights from input–output pairs, so behaviour is baked in, inference stays fast, and no vector database is needed.
- ✓ Fine-tuning's price is thousands of curated examples, GPU time, a full retraining round for every update, and the risk of catastrophic forgetting.
- ✓ Prompt engineering steers attention toward patterns already in the weights: instant and free, and structurally incapable of adding knowledge.
- ✓ Sequence the work cheapest-first — prompt, then retrieval, then fine-tuning — because each stage produces the evaluation the next one needs.

01 Why the same question gets different answers from different models

0:00

Everything a model knows sits in its weights, frozen when training stopped. Every improvement technique is a way of working around that one fact.

KEY POINTS

- Two models differ on the same question because their training corpora and cutoff dates differ.
- Parametric knowledge is distributed across weights — there is no record to edit, delete or audit.
- No model has seen your internal documents, regardless of how large it is.
- Three intervention points exist: the input, the weights, and the request.

The disambiguation problem

Ask 'who is Martin Keen?' and the model has to choose between an IBM engineer and the founder of a footwear company. Nothing in the question tells it which, so it picks one or blends both. Adding 'the Martin Keen who works at IBM' fixes it — and that is prompt engineering at its most basic. Note that this only works because the model already holds information about both people. If it knew about neither, no phrasing would help.

02 RAG: retrieval, augmentation, generation

1:29

The acronym is the architecture. Find relevant text, paste it into the prompt, then answer from it — with the model itself never modified.

KEY POINTS

- Retrieval → augmentation → generation; the acronym literally spells out the pipeline.
- The weights are never modified — RAG is entirely input-side.
- Grounding passages normally ship with an instruction to answer only from the supplied context.
- New knowledge goes live as soon as a document is indexed; there is no retraining step.
- If retrieval surfaces the wrong passages, generation is confidently wrong — debug retrieval first.

What actually reaches the model

The user typed one sentence. This is the prompt the model receives after augmentation. The explicit 'if it is not here, say so' clause is what converts a confident guess into an honest refusal.

Answer the question using only the context below.

If the context does not contain the answer, say that you do not know.

Cite the passage numbers you used.

<context>

[1] Q4 performance exceeded plan, with sales up across all regions...

[2] Quarterly sales rose 8% against the prior period, driven by renewals...

[3] The revenue recognition policy is described in appendix B...

</context>

Question: what was our company's revenue growth last quarter?

03 Embeddings: finding documents that share no words with the query

2:53

Both the question and every document become vectors, so relevance is measured as closeness in space rather than overlap in vocabulary.

KEY POINTS

- An embedding is a fixed-length vector whose geometry encodes meaning rather than spelling.
- Cosine similarity ranks by angle, so semantically similar text scores high with zero shared words.
- Documents are embedded once at index time; only the query is embedded per request.
- Chunk size is a real design parameter — too large blurs the vector, too small strips the context.
- Query and corpus must be embedded by the same model; vectors from different models are not comparable.

Scoring by meaning, not by words

The top match shares no content word with the query. The last candidate shares the word 'revenue' and still ranks bottom, which is precisely the behaviour a keyword index cannot produce.

```
import numpy as np

def cosine(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

# embed() maps text -> a fixed-length float vector (e.g. 768 or 1536 dims)
query = embed("what was our company's revenue growth last quarter?")

docs = [
    "Q4 performance exceeded plan, with sales up across all regions.",
    "Quarterly sales rose against the prior period.",
    "Office relocation is scheduled for March.",
    "The revenue recognition policy is described in appendix B.",
]

for score, doc in sorted(((cosine(query, embed(d)), d) for d in docs), reverse=True):
    print(f"{score:.3f} {doc}")
```

04 What RAG costs: latency per query, infrastructure forever

4:15

Retrieval is not free. It adds a round trip to every single request and a pipeline you have to operate indefinitely.

KEY POINTS

- RAG adds query embedding, vector search, and a longer input to every request's latency budget.
- You now run a vector database and an indexing pipeline as production infrastructure.
- A stale index yields confidently wrong answers; synchronisation is an ongoing obligation, not a setup task.
- Longer prompts also mean more input tokens billed on every call.
- RAG buys freshness and private-data coverage. It does not change behaviour, tone or format.

Where the time goes

The bracketed span is added to every query and cannot be cached away in the general case, because it depends on what the user just asked.

```
plain prompt :                               generate

RAG           : embed query → vector search → build prompt → generate
                  └────────── added to every query ─────────┘ (longer input)
```

05 Fine-tuning: continuing training on a narrow dataset

5:01

Take a model that already has broad competence and keep training it on examples of

the behaviour you want.

KEY POINTS

- Fine-tuning adjusts a pre-trained model; it is not training from zero.
- The dataset is supervised — input–output pairs demonstrating the behaviour you want.
- Your examples are the specification, so inconsistencies in them become learned behaviour.
- Inference requires no retrieval, so latency returns to plain-prompt levels.
- Knowledge lives in the weights, so there is nothing to keep synchronised at runtime.

The training file

Supervised fine-tuning data, one JSON record per line. Shown formatted here for readability — in the real file each record is a single line. Note that the target response carries structure as well as content: the model is learning the shape of a good answer, not only its facts.

```
{"messages": [  
  {"role": "user",  
    "content": "Scanner shows E-14 after the firmware update and tray 2 jams."},  
  {"role": "assistant",  
    "content":  
      "E-14 is a feed-sensor fault, and the firmware update resets\n  
      the sensor calibration.\n\n      Steps:\n  
      1. Power-cycle with tray 2 removed.\n  
      2. Re-run calibration from Service > Sensors.\n  
      3. Check the roller for debris.\n\n      If E-14 persists after calibration, escalate to hardware\n  
      with the sensor log attached."}  
]}
```

06 How the weights actually move

6:26

Supervised fine-tuning is a loss on predicted tokens, backpropagated into the weights,

repeated across the dataset.

KEY POINTS

- Loss is cross-entropy between the predicted token distribution and the target response.
- Backpropagation attributes the error to each weight; the optimiser steps them toward lower loss.
- Fine-tuning learning rates are deliberately tiny to avoid destroying pre-trained capability.
- Prompt tokens are masked out of the loss; only response tokens are scored.
- The result is changed processing, not just added facts — the model learns domain patterns.

The training loop, stripped down

Every fine-tuning framework wraps this, but this is what it wraps. The shift by one position is next-token prediction; the -100 labels are the masked prompt tokens that the loss ignores.

```
for batch in loader:
    out = model(input_ids=batch.input_ids, attention_mask=batch.mask)

    loss = cross_entropy(
        out.logits[:, :-1].flatten(0, 1),    # predictions, shifted
        batch.labels[:, 1:].flatten(),       # targets; -100 on prompt tokens
        ignore_index=-100,
    )

    loss.backward()        # attribute the error back to every weight
    optimizer.step()       # nudge them
    optimizer.zero_grad()
```

07 The bill for fine-tuning: data, GPUs, maintenance and forgetting

7:54

Four costs — and the fourth, losing general ability while gaining specialised ability, is the

one that catches teams out.

KEY POINTS

- Data collection usually dominates the cost, and it is human labour rather than compute.
- Full fine-tuning needs memory for weights, gradients and optimiser state at the same time.
- Every update is another training run; there is no way to incrementally add one fact.
- Catastrophic forgetting means specialisation degrades general capability.
- Always evaluate on a general benchmark you did not train on, not only on your domain set.
- LoRA and other PEFT methods freeze the base weights and train small adapters instead.

Catching forgetting before it ships

Illustrative numbers, not measurements — the shape is what matters. The domain score keeps climbing while the general score collapses, and a team watching only the first column would ship epoch 3 believing it was the best model they had.

	domain eval	general eval	
base model	41%	68%	
fine-tuned, epoch 1	72%	66%	← ship this
fine-tuned, epoch 2	76%	61%	
fine-tuned, epoch 3	78%	51%	← forgetting

08 Prompt engineering: aiming attention at capability that is already there

8:37

A prompt is not a request so much as a set of conditions. Change it and you change

which learned patterns dominate the output.

KEY POINTS

- Attention layers weigh the input against itself; prompt content decides what gets weighted.
- 'Think step by step' works by conditioning on a region of the training distribution, not by unlocking anything.
- Few-shot examples specify format and style more reliably than describing them in prose.
- Explicit task, scope, format and fallback each remove ambiguity the model would otherwise resolve on its own.
- No infrastructure, no training, and a feedback loop measured in seconds.

'Is this code secure?' versus an engineered prompt

The first version leaves the model to decide what 'secure' covers, how deep to look, what format to answer in, and what to do when it finds nothing. The second decides all four. Same model, same weights, same code under review.

before

Is this code secure?

after

Review the Python function below for security defects.

Scope: injection, authentication and authorisation, secrets handling, unsafe deserialisation, and input validation. Ignore style and performance.

Work through the data flow from untrusted input to each sink before answering.

For every finding report:

- severity (high / medium / low)
- the exact line
- why it is exploitable, concretely
- a fix

If a category has no findings, say so explicitly rather than omitting it.

FUNCTION:

<code here>

09 Where prompt engineering stops

10:50

No phrasing adds information the weights never held, and finding a good prompt is empirical rather than derivable.

KEY POINTS

- Prompting cannot add knowledge that is not in the weights.
- It cannot correct stale or wrong parametric knowledge either.
- Pushed past its knowledge, a model produces fluent invention rather than refusing.
- Prompt quality is discovered empirically — the leverage is in the eval harness, not the wording.
- If the missing ingredient is facts, the fix is retrieval, not a better prompt.

A minimal prompt eval harness

Thirty lines and it changes how prompt work is done. Each case pairs an input with a programmatic check. Compare candidate prompts on the same cases and the argument about which is better stops being a matter of opinion.

```
CASES = [
    ("def f(u): os.system('ping ' + u)",
     lambda out: "injection" in out.lower() and "high" in out.lower()),
    ("def add(a, b): return a + b",
     lambda out: "no findings" in out.lower(),          # must not invent bugs
    ]

def score(template):
    passed = sum(
        check(model.generate(template.format(code=code)))
        for code, check in CASES
    )
    return passed / len(CASES)

for name, template in CANDIDATE_PROMPTS.items():
    print(f"{name:<24} {score(template):.0%}")
```

10 Combining all three

11:33

In production these are layers rather than alternatives, each covering a gap the other two leave open.

KEY POINTS

- Retrieval for facts, fine-tuning for behaviour, prompting for the request itself.
- A legal system uses all three: case retrieval, firm-specific style, per-request formatting.
- Prompt engineering is flexible and immediate but cannot extend knowledge.
- RAG extends knowledge and keeps it current, at the price of latency and infrastructure.
- Fine-tuning enables deep domain expertise, at the price of data, compute and maintenance.
- Work cheapest-first: prompt, then retrieval, then fine-tuning — each stage builds the eval the next one needs.

Symptom to lever

Diagnose from the failure, not from the technique you already wanted to use. Two of these entries point at fine-tuning for reasons that have nothing to do with knowledge, which is the distinction people most often collapse.

symptom	reach for
"it doesn't know our internal facts"	RAG
"it's citing last year's numbers"	RAG (and check index freshness)
"it ignores half my instructions"	prompt engineering
"the output format keeps drifting"	prompting → fine-tuning at scale
"it doesn't sound like our house style"	fine-tuning
"it's too slow"	fine-tuning (removes the retrieval hop)
"it invents facts under pressure"	RAG + an explicit refusal clause

Glossary

Parametric knowledge

Information encoded implicitly in a model's weights during training. It cannot be listed, edited or deleted directly — only probed by asking the model.

Knowledge cutoff

The date at which a model's training data ends. Nothing after it exists in the weights, no matter how significant.

RAG (Retrieval Augmented Generation)

Searching a corpus for passages relevant to a query, inserting them into the prompt, and generating an answer from them. The model itself is never modified.

Vector embedding

A fixed-length list of numbers representing a piece of text, arranged so that texts with similar meaning produce nearby vectors.

Cosine similarity

The cosine of the angle between two vectors, used to score relevance. It ignores magnitude, so a long document and a short phrase about the same thing still match.

Vector database

A store built for fast nearest-neighbour search over embeddings. It is the piece of infrastructure RAG obliges you to run and keep in sync.

Chunking

Splitting documents into passages before embedding them. Chunk size and boundaries strongly affect whether retrieval can find anything.

Grounding

Constraining a model to answer from supplied text rather than from memory, usually by instruction. It is what makes a retrieved answer checkable.

Fine-tuning

Continuing training on a pre-trained model with a focused dataset, so the desired behaviour ends up stored in the weights.

Supervised fine-tuning (SFT)

Fine-tuning on explicit input–output pairs, where each example shows a request and the response the model should have produced.

Backpropagation

The algorithm that works out how much each weight contributed to the error, so the optimiser knows which direction to move it.

Cross-entropy loss

The standard training objective for language models: it is large when the model assigned low probability to the token that was actually correct.

Catastrophic forgetting

Loss of general capability during specialised training, as the weights drift away from the configuration that supported it.

LoRA / PEFT

Parameter-efficient fine-tuning: freeze the base model and train small adapter matrices instead. Cheaper to train, cheaper to store, and reversible.

Attention mechanism

The component in each transformer layer that weighs parts of the input against one another, deciding what influences the output most.

Few-shot prompting

Including worked examples in the prompt to demonstrate the desired output rather than describing it.

Context window

The maximum amount of text a model can attend to at once. It bounds how much retrieved material a RAG system can supply.

Inference

Running a trained model to produce output, as opposed to training it. RAG adds work here; fine-tuning moves work out of it.

Check yourself

Answer first, then open each one.

RAG and fine-tuning both let a model use information it was never originally trained on. Why does only one of them leave you operating infrastructure?

RAG resolves knowledge at query time, so the corpus must stay embedded, indexed and synchronised in a vector database for as long as the system runs. Fine-tuning resolves it at training time and writes the result into the weights, so at inference there is nothing external to consult — the cost moves from ongoing operations to periodic retraining.

A keyword search for 'revenue growth' returns nothing, yet the RAG system surfaces a document titled 'Q4 performance'. What made that possible, and what would break it?

Both the query and the document were converted to vector embeddings, and relevance was scored by cosine similarity, which measures closeness in meaning rather than overlap in words. It breaks if the query and the corpus are embedded by different models — the vectors then occupy incomparable spaces — or if chunking split the relevant passage so that neither half carries the meaning.

Your fine-tuned support model handles tickets far better than the base model, but has started giving noticeably worse answers to ordinary general questions. What happened, and what should you have been measuring?

Catastrophic forgetting: training hard on a narrow distribution pulled the weights away from the configuration supporting general capability. You should have run a held-out general-capability evaluation at every checkpoint alongside the domain evaluation, so the falling general score was visible before you shipped. A domain-only dashboard cannot show this by construction.

Why does 'think step by step' improve output when the weights are byte-for-byte identical either way?

The instruction conditions the model on a region of its training distribution where methodical reasoning preceded correct results, so attention weights shift toward those patterns. No capability is added — existing capability is selected. This is also why the effect is unreliable: it depends on what the training data associated with such phrasing.

A model confidently states something wrong about a policy that exists only in your internal wiki. Why is rewriting the prompt the wrong fix?

The information was never in the weights, so there is no pattern for a prompt to activate. Prompting redistributes attention over what the model holds; it cannot introduce what is absent. The fix is retrieval — put the policy text into the context — plus an explicit instruction to say 'I don't know' when the context is silent.

Why is it better to build a prompt evaluation harness before fine-tuning rather than after?

Two reasons. Prompting may solve the problem outright, at no cost, which you cannot know without measuring. And if it does not, the harness is the only instrument that can tell you whether fine-tuning improved things — without a pre-existing baseline on fixed cases, 'the fine-tuned model seems better' is an impression, not a result.

Where to go next

- Build the twelve-line RAG loop against a folder of your own PDFs, then vary chunk size across 200, 500 and 1000 tokens and watch retrieval quality change with nothing else touched.
- Add a reranking stage: retrieve 50 candidates by vector search, score them with a cross-encoder, keep the top 4, and measure whether answer quality justifies the extra latency.
- Instrument each stage of a RAG query separately — query embedding, vector search, generation — and find where the time actually goes before optimising anything.
- Fine-tune a small open model with LoRA on a few hundred examples, and evaluate at every checkpoint on both a domain set and a general benchmark to watch forgetting happen in real numbers.
- Read the LoRA paper (Hu et al., 2021) for the argument that weight updates during adaptation have low intrinsic rank, which is why small adapters recover most of full fine-tuning's benefit.
- Compare RAG against long-context prompting on the same task: paste the entire document set into the context window and measure cost, latency and accuracy against retrieval over the same corpus.

- Write ten fixed test cases for a prompt you already use in production, score your current wording against two rewrites, and see whether the version you assumed was best actually is.
-

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.